AI-Driven SRE

AI와 함께 만드는 쿠버네티스 운영 표준

Command Guardrails 패턴과 4계층 문서 체계의 탄생

바이라인네트워크 | AI 시대를 준비하는 쿠버네티스 - 클라우드 네이티브의 재발견

2026. 03. 24

조 훈 (Hoon Jo) | CNCF Ambassador | AI & Cloud Native Engineer @MegazoneSoft

https://github.com/SysNet4Admin

https://www.linkedin.com/in/hoonjo

문제 인식

Problem Recognition

이런 (가정) 미션이 주어졌습니다

MegaBird — 대량 메시지 발송 플랫폼

V1 → V2 EKS 마이그레이션

- EKS 25노드, 429파드, 9개 노드그룹
- 15개 Helm 릴리스
- Kafka (ZooKeeper → KRaft)
- Redis → Valkey 전환
- 옵저버빌리티 스택 전체 재구축
- 46개 알림 규칙, 7개 알림 경로

제약 조건

V1 프로덕션 절대 중단 불가

24/7 메시지 발송 중

1인 SRE가 DEV + PROD 동시 구축

아키텍처 현대화 포함

단순 복제가 아닌 전면 재설계

전통적 접근의 한계

```
전통적 SRE 워크플로우:
  Wiki 읽기 → 터미널에서 수작업 → 결과를 Wiki에 기록

문제:
  ├── IaC: 선언적이지만 "판단"은 못 함
  ├── Runbook: 상황마다 달라서 그대로 실행 불가
  └── 자동화 스크립트: 유지보수가 또 다른 부담
```

핵심 맥락

"규모가 큰 프로젝트라서 AI를 썼다"가 아니라

"1인이라는 제약 속에서

AI를 동료로 삼는 방법을 찾아야 했다"

"만약 runbook을 이해하고, 상황에 맞게 판단하고,

실행까지 하는 동료가 있다면?"

→ AI 코파일럿 (Claude Code) 을 전면 활용하기로 결정

4계층 문서 체계

Four-Layer Document Architecture

Layer 1: work-plans — 사람이 읽는 계획서

```
work-plans/
├── 00-main/          # 마스터 플랜 + DEV/PROD 런북
├── 10-infrastructure/  # EKS, 스토리지, AWS 연동
├── 20-platform/       # Prometheus, Loki, Tempo, ArgoCD
├── 30-data/           # Valkey, Kafka, Kafka Streams
├── 40-transition/     # 무중단 전환, Weighted Routing
└── 90-future-projects/
```

- 사람이 읽고 판단하기 위한 상세 계획서 (한국어)
- 각 컴포넌트의 Why + How + 트레이드오프 문서화

문제 발견

이 문서를 AI에게 그대로 주면?

- 너무 길고, 맥락이 많고, 한국어 혼재
- AI가 핵심을 놓치거나 엉뚱한 부분에 집중

"사람이 보는 문서와 AI가 보는 문서는 달라야 했습니다"

Layer 2: claude-context — AI를 위해 증류한 문서

```
claude-context/
├── 00-project-summary.md      # 프로젝트 1페이지 요약
├── 01-environment-values.md   # 모든 환경 값 (복붙 가능)
├── 02-document-map.md         # 문서 네비게이션 맵
├── 03-current-status.md       # 현재 배포 상태 (라이브)
└── 99-execution-guide.md      # 실행 순서 + 체크리스트
```

- work-plans의 핵심만 영어로, 최소한으로 증류(distill)
- AI가 프로젝트 상태를 즉시 파악하는 대시보드 역할

비유: work-plans = 교과서, claude-context = 시험 범위 요약노트

또 다른 문제 발견

AI가 맥락은 이해하는데,
실행할 때 자기 나름대로 명령어를 만들어냄

- "Loki 설치해줘" → AI가 helm install 자체 생성
- 버전 다름, values 누락, 네임스페이스 틀림...

"AI가 이해하는 것과 올바르게 실행하는 것은 다른 문제였습니다"

Layer 3: command-guardrails — AI의 행동 제어

```
command-guardrails/
├── dev/
│   ├── 00-prerequisites/   # KMS, S3, EFS
│   ├── 10-infrastructure/  # EKS, 노드그룹, 애드온
│   ├── 20-platform/        # Helm (Prometheus, Loki...)
│   ├── 30-data/            # Valkey, Kafka
│   ├── 40-transition/      # 알림, 도메인, 트래픽 전환
│   └── 80-rollback/        # 롤백 절차
└── prod/
    └── (동일 구조, PROD 값 + DEV 교훈 반영)
```

- 각 파일 = 단계별 실행 가이드 (마크다운 안의 bash 블록)
- V1 보호 규칙: get / describe / logs 만 허용
- 순서 강제: 00 → 10 → 20 → 30 → 40

핵심 설계 원칙

"AI 시대의 인프라 문서는 사람이 읽는 문서가 아니라,
AI가 실행하는 문서다"

그런데 또 문제 발견

guardrail에 "helm upgrade prometheus-stack"만 적으면
AI가 --set 플래그를 창의적으로 만들어냄
→ 경우의 수 폭발, 재현 불가능한 배포

**"무엇을 실행하라는 알려줘도,
정확히 어떤 값으로는 보장하지 못했습니다"**

Layer 4: helm-values — IaC에 가까워지다

왜 이 계층이 필요해졌는가 — 실제 사고

Mimir 버전 사고

guardrail에 helm upgrade mimir만 적고 --version 미지정

→ 5.8.0 이 6.0.5로 자동 업그레이드 → CrashLoop

→ 교훈: 버전을 값 파일에 고정해야 함

PromQL false-positive

AI가 relativeTimeRange(600s) + for(5m) 조합 사용

→ 순간 스파이크 false alarm → 7개 규칙 일괄 수정

→ 교훈: 알림 파라미터도 설정 파일로 관리

최종 형태

```
helm upgrade loki grafana/loki \
  --version 6.30.0 \        # 버전 고정
  -f helm-values/loki.yaml \ # 값 고정 (IaC)
  -n monitoring \           # 네임스페이스 고정
  --wait --timeout 10m      # 동작 고정
```

```
helm-values/  (15개 파일)
├── prometheus-stack.yaml
├── loki.yaml
├── mimir.yaml
├── tempo.yaml
├── opensearch.yaml
├── argo-cd.yaml
└── ... (8개 더)
```

4계층이 처음부터 설계된 게 아니라, 각 단계에서 AI의 한계를 발견하고 보완하면서 자연스럽게 진화한 결과

4계층 전체 조감도

사람의 판단
(추상적)

Layer 1: work-plans
사람이 읽는 계획서
Why + How + 트레이드오프

Layer 2: claude-context
AI를 위한 증류 문서
프로젝트 상태, 환경 값, 실행 가이드

▼

Layer 3: command-guardrails
AI의 행동 제어
단계별 실행 가이드, 순서 강제, 권한 제한

AI의 실행
(구체적)

Layer 4: helm-values / IaC
값의 고정
해석의 여지 제거, 재현 가능한 배포

위에서 아래로: 추상 → 구체 | 자유도 감소 | 재현성 증가

운영 전략

Operational Strategy

AI를 위한 문서가 팀의 자산이 되다 — GitAIOps

계층	AI에게는	사람 (후임자/신규 인원)에게는
work-plans	(직접 참조하지 않음)	의사결정 히스토리
claude-context	프로젝트 맥락 파악	온보딩 문서 / 대시보드
command-guardrails	실행할 명령어 가이드	그대로 따라하는 절차서
helm-values	해석 없이 적용할 값	Single Source of Truth

```
Git  (버전 관리 + 리뷰 + 변경 이력)
+ AI (맥락 이해 + 가이드 실행 + 결과 검증)
─────────────────────────────────────
= GitAIOps
```

GitOps

선언적 인프라, Git이 Single Source of Truth

GitAIOps

+ AI가 문서를 이해하고 실행하는 계층

- 누가 실행해도 같은 결과 → 일관성 보장
- 변경 이력이 남으니 → 감사 추적 가능

"AI를 위해 만든 문서가, 결국 팀 전체의 운영 표준이 되었다."

AI에게 맡기는 것의 경계

AI에게 맡기기 좋은 것	사람이 판단해야 하는 것
클러스터 상태 점검, 문서 vs 실제 차이 탐지	차이의 원인이 의도적인지 실수인지
DEV에서 검증된 패턴을 PROD에 적용	PROD 적용 시점과 순서 결정
작업 결과를 guardrail/context에 반영	아키텍처 의사결정
로그 분석, 에러 패턴 탐색	"이 에러는 무시해도 되는가"
46개 알림 규칙 일괄 생성/수정	알림 임계값, 심각도 분류
—	V1 프로덕션 영향 판단, 롤백 결정

"자동 운전이 아니라 운전 보조. 핸들은 사람이 잡되, 사각지대를 AI가 잡아준다."

AI의 기억은 대화가 아니라 파일에 있다

Memory 시스템

```
[분석 수행] 35개 규칙 전수 분석
  → 커버리지 갭 5개, P0/P1/P2 분류 완료

[compaction 발생] 원본 사라짐
  → 요약만 남음: "P0/P1/P2 분류 완료"

[문제] "P1 항목이 뭐였지?" → 대답 불가
```

역할	설명
맥락 보존	compaction에서 살아남는 영속 기억
금지 사항 누적	"하지 마라"의 축적 (Guardrail 보완)
작업 상태 추적	대화가 끊어져도 진행 상황 이어짐
콜드스타트 방지	매 세션 재탐색 없이 즉시 시작

```
memory/
├── MEMORY.md                  # 인덱스 (200줄, 자동 주입)
├── alert-gap-analysis.md      # 토픽 파일 (필요 시 로드)
├── osd-lessons.md             # 토픽 파일 (필요 시 로드)
└── argocd-lessons.md          # 토픽 파일 (필요 시 로드)
```

관리하지 않으면 독이 된다

- 200줄 초과 → 경고 없이 잘림 (hard truncation)
- 오래된 기억 → AI가 잘못된 근거로 판단
- 지시 준수율: 200줄 이하 92% → 400줄+ 71%

→ MEMORY.md는 인덱스만, 상세는 토픽 파일로 분리

"4계층과 같은 원리: 증류 + 분리 + 필요할 때만 로드"

데모

Demo

Live Demo

옵저버빌리티 감사 (Observability Audit)

```
demo/
├── CLAUDE.md                    # AI 행동 규칙
├── claude-context/
│   └── 00-cluster-summary.md   # 클러스터 맥락
├── command-guardrails/
│   └── observability-audit.md   # 실행 가이드
└── memory/
    └── MEMORY.md                # 누적 기억
```

① CLAUDE.md — Ground Rules
READ-ONLY만 허용, 수정 명령 금지
kubectl get/describe/logs만 허용

② claude-context — 클러스터 맥락
EKS 2노드, 옵저버빌리티 스택 구성
Prometheus, Loki, Tempo, ArgoCD, Istio

③ command-guardrails — 9단계 감사
Step 1~9 순서대로만 실행
결과를 Audit Report 형식으로 출력

④ memory — 누적 학습
이전 감사에서 발견된 이슈 기록
매 세션 콜드스타트 방지

github.com/sysnet4admin/talks/.../demo

Demo: Command Guardrail 구조

9단계 감사 절차 — AI가 순서대로 실행

Step 1 Cluster Health Check
kubectl get nodes, namespaces

Step 2 Helm Release Audit
helm list -A

Step 3 Pod Health — Monitoring
kubectl get pods -n monitoring

Step 4 Prometheus Targets
ServiceMonitor / PodMonitor

Step 5 Log Pipeline
Fluentd DaemonSet + Loki

Step 6 Tracing Pipeline
Tempo + OTel Collector

Step 7 ArgoCD & Rollouts
Applications Sync/Health

Step 8 Istio Service Mesh
istio-system + Kiali

Step 9 Grafana Alert Rules
PrometheusRules

Output: Audit Report

```
## Audit Report
Date: 2026-03-24

### Findings
| # | Severity  | Component | Issue |
|---|-----------|-----------|-------|
| 1 | CRITICAL  | Loki      | ...   |
| 2 | WARNING   | Fluentd   | ...   |
| 3 | INFO      | Tempo     | ...   |

### Next Steps (사람이 판단)
- CRITICAL: 즉시 조치 여부 판단
- WARNING: 일정 잡아서 조치
- INFO: 기록만 하고 모니터링
```

"AI는 guardrail의 명령만 실행하고,
사람은 결과를 보고 판단한다"

변화와 제안

Changements & Recommendations

생산성 변화

작업	Before (수작업)	After (AI 코파일럿)
DEV 환경 전체 구축	추정 1~2주	~2일
PROD 환경 구축 (guardrail 재사용)	추정 1주	~1일
클러스터 감사/점검	수시간	30분
46개 알림 규칙 구축	수일	수시간

"AI가 사람을 대체한 게 아니라, 1인의 커버리지를 5인분으로 확장했다"

조직 적용 로드맵

Level 0 1주	기존 runbook을 구조화된 markdown으로 정리
Level 1 2~4주	AI 코파일럿으로 DEV 환경에서 실험 시작
Level 2 1~2개월	Context + Guardrails + Memory 3계층 체계 구축
Level 3 지속	DEV→PROD 패턴 정착, helm-values(IaC) 추가

3가지 제안

1

문서의 목적에 따라 구분 for human & AI

인간을 위한 문서와 Ai가 읽는 문서는 형태와 구조가 달라야함

2

DEV → PROD 패턴

DEV에서 검증한 guardrail을 PROD에 재사용하라

PROD guardrail에는 "DEV 교훈" 섹션이 자동 추가됨

3

Memory에 기록하라

대화는 사라져도 파일은 남는다

진행 중인 작업과 분석 결과를 memory에 남기면

다음 대화에서도 맥락이 이어진다

전통적 SRE	AI-Driven SRE
kubectl 실행자	AI가 실행할 가이드 설계자
Runbook 작성자	Guardrails 아키텍트
수동 감사/점검	AI 감사 결과 검증자
반복 작업 수행	판단과 의사결정에 집중
1인 = 1환경	1인 = N환경

AI-Driven SRE는

더 적은 사람이 더 많은 시스템을 운영하는 것이 아니라,

같은 사람이 더 나은 판단에 집중하는 것이다.

(Cloud & AI Native, k8s Engineer, linkedin)